

LABORATORY PROJECT REPORT

COLOR DETECTION AND ANALYSIS

EXPERIMENT 9

Section 1

Group 2

Semester 1 2025/2026

NO	NAME	MATRIC NO
1.	ADAM NURUDDIN BIN HELMI	2215783
2.	NUR FATIHAH HANNANI BINTI HASMAYADI	2227062
3.	NUR SYAHZMAN AZIQ BIN MOHD SHAHMADI	2310037

Date of submission: 17th December 2025

Abstract

This experiment focuses on image-based color detection using an AI vision system and microcontroller integration. A Gravity HuskyLens AI Vision Camera was interfaced with an Arduino Uno to detect predefined colors and provide real-time visual feedback through an RGB LED (KY-016). The HuskyLens camera was trained using its built-in color recognition mode to identify red, green, and blue objects, each assigned a unique ID. Upon successful detection, the Arduino processes the received data and activates the corresponding RGB LED color using Pulse Width Modulation (PWM). This experiment demonstrates the application of image and video interfacing techniques, sensor–actuator integration, and embedded system programming without the need for external image processing software. Through this laboratory exercise, students gain hands-on experience in AI-assisted vision sensing, serial communication, and real-time hardware response based on visual input, reinforcing key concepts in mechatronics system integration.

Table of Contents

1.0 Introduction

2.0 Materials and Equipment

2.1 Electronic Components

2.2 Equipment and Tools

3.0 Experimental Setup

4.0 Methodology

5.0 Data Collection

6.0 Data Analysis

7.0 Results

8.0 Discussion

9.0 Conclusion

10.0 Recommendations

11.0 References

1.0 Introduction

Image and video interfacing play an important role in modern mechatronic systems, particularly in applications involving automation, robotics, and intelligent sensing. Traditional color detection systems often rely on dedicated color sensors, which may be sensitive to lighting conditions and require extensive calibration. With the advancement of AI vision technology, compact vision sensors such as the HuskyLens AI Camera provide a more robust and flexible alternative for visual recognition tasks.

In this experiment, the Gravity HuskyLens AI Vision Camera is used as an image input device to detect specific colors using its internal AI based color recognition algorithm. The detected color information is transmitted to an Arduino Uno through serial communication. An RGB LED (KY-016) is used as an output indicator to visually represent the detected color in real time.

This laboratory exercise emphasizes the integration of an AI vision sensor with a microcontroller, focusing on color recognition, UART communication, and actuator output control. The system provides a simple yet effective demonstration of image-based sensing without requiring complex external software or high computational resources. The skills developed in this experiment are relevant to applications such as automated sorting systems, smart traffic indicators, and human–machine interaction systems.

2.0 Materials and Equipment

2.1 Electronic Component

- **Arduino Uno** – Acts as the main microcontroller to process data and control output devices
- **Gravity HuskyLens AI Vision Camera** – Performs AI-based color recognition
- **KY-016 RGB LED Module** – Provides visual indication of detected colors
- **Jumper Wires** – Used to establish electrical connections
- **Breadboard** – For circuit assembly and organization

2.2 Equipment and Tools

- **Arduino IDE** – Used to write and upload the program to the Arduino Uno
- **USB Cable** – For programming and powering the Arduino Uno
- **HuskyLens Built-in Display & Buttons** – Used for color training and mode selection

3.0 Experimental Setup

The experimental setup consists of interfacing the HuskyLens AI Camera with an Arduino Uno using UART serial communication and connecting an RGB LED as an output indicator.

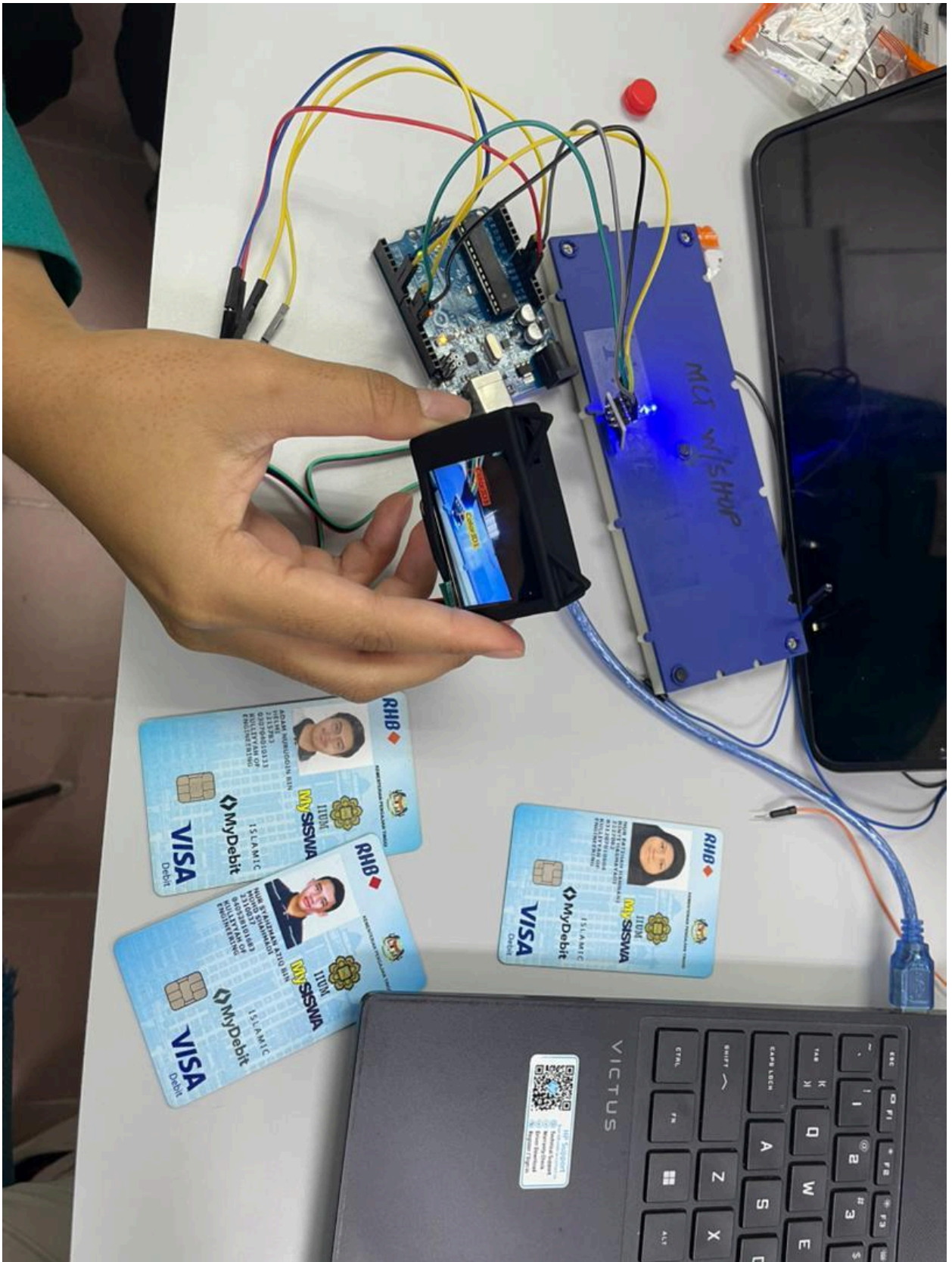
1. The HuskyLens camera is connected to the Arduino Uno using SoftwareSerial communication.
2. The HuskyLens VCC and GND pins are connected to the Arduino 5V and GND respectively.
3. The TX and RX pins of the HuskyLens are connected to Arduino digital pins D5 and D6.
4. The KY-016 RGB LED module is connected to the Arduino using PWM-capable pins:
 - Red pin → D9
 - Green pin → D10
 - Blue pin → D11
5. The Arduino Uno is powered through a USB connection to the computer.
6. The HuskyLens is set to Color Recognition Mode, and three colors (red, green, and blue) are trained using the learning button.

4.0 Methodology

The experiment was conducted following a systematic procedure to ensure accurate color detection and reliable hardware response.

4.1 Detailed steps

1. The HuskyLens AI camera was powered on and switched to **Color Recognition Mode** using the onboard buttons.
2. Three different colored objects (red, green, and blue) were individually trained using the HuskyLens learning button, with each color assigned a unique detection ID.
3. The Arduino Uno was connected to the computer, and the program code was written and uploaded using the Arduino IDE.
4. The program initializes serial communication between the Arduino and HuskyLens using SoftwareSerial.
5. The Arduino continuously requests detection data from the HuskyLens.
6. When a color is detected, the HuskyLens sends the corresponding color ID and position data to the Arduino.
7. Based on the received ID:
 - Red object → Red LED turns ON
 - Green object → Green LED turns ON
 - Blue object → Blue LED turns ON
8. If no object or an untrained color is detected, all RGB LED outputs are turned OFF.
9. The detected color ID and coordinates are displayed on the Serial Monitor for monitoring and debugging.
10. The system is tested under different object positions and lighting conditions to observe detection consistency.



4.2 Programming implementation

Arduino codes

```
#include <SoftwareSerial.h>    // Library to create software-based serial communication
```

```
#include "HUSKYLENS.h"        // Library for HuskyLens AI camera
```

```
// Create HuskyLens object
```

```
HUSKYLENS huskylens;
```

```
// Create SoftwareSerial object for HuskyLens communication
```

```
// RX = pin 5, TX = pin 6
```

```
SoftwareSerial mySerial(5, 6);
```

```
// KY-016 RGB LED pins connected to Arduino PWM pins
```

```
int redPin = 9;                // Red LED pin
```

```
int greenPin = 10;            // Green LED pin
```

```
int bluePin = 11;            // Blue LED pin
```

```
void setup() {
```

```
    // Initialize serial communication for debugging
```

```
    Serial.begin(9600);
```

```
    // Initialize serial communication for HuskyLens
```

```
    mySerial.begin(9600);
```



```
// Set RGB LED pins as output

pinMode(redPin, OUTPUT);

pinMode(greenPin, OUTPUT);

pinMode(bluePin, OUTPUT);


// Try to establish communication with HuskyLens

// The loop will continue until HuskyLens is detected

while (!huskylens.begin(mySerial)) {

    Serial.println("HuskyLens not connected!");

    delay(1000);          // Wait before retrying

}


// Indicate that HuskyLens is ready

Serial.println("HuskyLens Ready.");

}


// Function to turn OFF all RGB LED colors

void ledOff() {

    analogWrite(redPin, 0);    // Turn off red LED

    analogWrite(greenPin, 0);  // Turn off green LED

    analogWrite(bluePin, 0);   // Turn off blue LED

}
```

```
// Function to set a specific RGB LED color

// r, g, b values range from 0 to 255

void setColor(int r, int g, int b) {

    analogWrite(redPin, r);    // Set red LED brightness

    analogWrite(greenPin, g);  // Set green LED brightness

    analogWrite(bluePin, b);   // Set blue LED brightness

}


void loop() {

    // Request data from HuskyLens

    // If request fails, exit the loop iteration

    if (!huskylens.request()) {

        Serial.println("Request failed");

        return;

    }


    // Check if HuskyLens has detected any object

    if (huskylens.available()) {


        // Read the detected result from HuskyLens

        HUSKYLENSResult result = huskylens.read();
```

// Display detected color ID and position on Serial Monitor

Serial.print("Detected Color ID: ");

Serial.print(result.ID);

Serial.print(" at (X: ");

Serial.print(result.xCenter);

Serial.print(", Y: ");

Serial.print(result.yCenter);

Serial.println(")");

// Control RGB LED based on detected color ID

if (result.ID == 2) {

// If color ID 2 is detected, turn on RED LED

Serial.println("Color 1 detected! -> RED");

setColor(255, 0, 0);

}

else if (result.ID == 3) {

// If color ID 3 is detected, turn on GREEN LED

Serial.println("Color 2 detected! -> GREEN");

setColor(0, 255, 0);

}

else if (result.ID == 1) {

// If color ID 1 is detected, turn on BLUE LED

Serial.println("Color 3 detected! -> BLUE");

```

    setColor(0, 0, 255);
}

else {

    // If detected ID does not match trained colors, turn off LED

    ledOff();

}

}

else {

    // If no object is detected, turn off RGB LED

    Serial.println("No object detected");

    ledOff();

}

// Small delay to stabilize detection and reduce serial spam

delay(200);

}

```

5.0 Data Collection

COLOR	DETECTED ID	X-CENTER	Y-CENTER	RGB LED OUTPUT
BLUE	1	161	120	BLUE ON
RED	2	152	109	RED ON
GREEN	3	172	128	GREEN ON

6.0 Data Analysis

During the task, the HuskyLens AI Vision Camera successfully detected the three trained colours and sent the detection results to the Arduino Uno via UART communication using the SoftwareSerial library. Blue color was received as Color ID 1, red color was received as Color ID 2, and finally green color was received as Color ID 3. The correct RGB LED activation confirmed that the communication between HuskyLens and Arduino Uno was working reliably.

The X-center and Y-center values recorded during each trial showed slight variations, which is expected due to changes in object position and small movements during detection. However, these variations did not affect the color identification process, as the code only relied on the detected Color ID to make decisions.

7.0 Results

The color detection system using the Gravity HuskyLens AI Vision Camera and Arduino Uno operated successfully throughout the experiment. The HuskyLens consistently detected the three trained colors and sent the corresponding Color ID along with the X-center and Y-center coordinates of the object to the Arduino Uno via UART communication. The detected data was displayed on the Serial Monitor and used to control the RGB LED as visual output.

The recorded X-center and Y-center values showed small variation between trials due to the movement and position of the objects in front of the camera. However, these variations did not affect color recognition or LED response.

8.0 Discussion

Two different approaches were used to successfully complete the colour detection experiment: an advanced approach using the Gravity HuskyLens AI Vision Camera interfaced with an Arduino Uno, and a basic approach using a dedicated colour sensor (e.g., TCS3200) coupled with Arduino and Python serial communication. RGB colour values may be directly and affordably measured using the Colour Sensor (TCS3200) technique. However, this method's precision and consistency are quite sensitive to changes in lighting conditions and heavily rely on careful sensor calibration. The raw RGB measurements can be drastically changed by changes in ambient light or the distance between the sensor and the object, requiring intricate Python software normalisation. The speed of the sensor's analog-to-digital conversion, the serial connection baud rate, and the host PC's Python script processing speed all affect how quickly the system responds.

The Gravity HuskyLens AI Vision Camera, on the other hand, offered a more reliable and reliable colour identification system. An integrated AI system used by the HuskyLens is initially calibrated for up to three distinct colours through a "learning" process. Compared to the raw

sensor data, this pre-processing capacity makes it intrinsically less sensitive to slight variations in illumination. Additionally, the HuskyLens provides spatial context that is not possible with a simple colour sensor by outputting processed data, such as the object's width, height, x, and y coordinates, as well as the detected Colour ID. The Arduino Uno's sole hardware serial port necessitated the usage of the SoftwareSerial library for the whole arrangement, which connected the TX/RX pins to D4/D5, respectively.

In conclusion, especially in situations where illumination could not be precisely regulated, the AI Vision Camera (HuskyLens) produced the most accurate and reliable findings. The final detection performance was better since the onboard AI could generalise colour characteristics instead of depending on raw light intensity measurements, even if the setup and library needs (DFRobot_HuskyLens) were a little more complicated.

9.0 Conclusion

Through two experimental components, the goal of creating and evaluating a colour-detecting system was effectively accomplished. The Colour Sensor technique is a great way to learn about sensor interface and serial data processing. It shows that resilience in the real world needs considerable software compensation for basic, raw data collection. On the other hand, a more advanced option that integrates machine vision concepts is offered by the Gravity HuskyLens AI Vision Camera. The HuskyLens system demonstrated its superiority through built-in algorithms that control environmental factors and supply crucial object localisation data, as evidenced by the evaluation of important findings, which showed that it provided the most precise and consistent colour recognition. The experiment demonstrated that including an AI vision sensor provides a definite advantage over a simple, raw-data colour sensor for industrial or dynamic applications.

10.0 Recommendations

1. Improved Algorithms and Calibration

Moving the colour detecting algorithm from the RGB space to the HSV (Hue, Saturation, Value) colour space would be a major improvement for the colour sensor. By reducing the dependence of colour identification (Hue) on light intensity (Value), this conversion strengthens the detection against changing lighting conditions. Additionally, the initial accuracy would be significantly improved by putting in place an automated, multi-point calibration procedure that averages many measurements for reference colours.

2. Sensor Configuration and System Optimisation

The Arduino-to-PC serial connection for the colour sensor should be optimised by utilising a higher baud rate (e.g., 9600) in order to increase the system's reaction time. Using an Arduino board with many hardware serial ports (such as the Arduino Mega) would improve data transmission speed and reliability for the HuskyLens system by doing away with the requirement for SoftwareSerial. The system's capacity to precisely identify and validate items in a regulated field of vision may also be improved by investigating sensor fusion, which combines the HuskyLens's object identification with a proximity or ultrasonic sensor.

12.0 Acknowledgement

Special thanks to ZULKIFLI BIN ZAINAL ABIDIN & WAHJU SEDIONO for their guidance and support during this experiment.

Student's Declaration

Certificate of Originality and Authenticity

We hereby certify that we are collectively responsible for the work presented in this report. The content reflects our original work, except where specific references or acknowledgements have been made. No part of this report has been completed or submitted by individuals or sources not identified within.

Furthermore, we confirm that this report is the result of a collaborative effort, with contributions made by all group members. The extent of each individual's contribution is documented within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Name: ADAM NURUDDIN BIN HELMI	Read	[/]
Matric Number: 2215783	Understand	[/]
Signature: <i>ADAM</i>	Agree	[/]

Name: NUR FATIHAH HANNANI BINTI HASMAYADI	Read	[/]
Matric Number: 2227062	Understand	[/]
Signature: <i>FAT</i>	Agree	[/]

Name: NUR SYAHZMAN AZIQ BIN MOHD SHAHMADI	Read	[/]
Matric Number: 2310037	Understand	[/]
Signature: <i>AZIQ</i>	Agree	[/]

